

# Semáforo Vehicular y Peatonal con Máquina de Estados Finitos

*Reporte de práctica — Sistemas Programables*

## 1. Descripción del proyecto

Este proyecto implementa el control de un semáforo vehicular y peatonal utilizando una máquina de estados finitos (FSM) programada en Arduino UNO R4 WiFi. El sistema alterna automáticamente entre los estados de un semáforo convencional (verde, amarillo y rojo vehicular) e incorpora un botón peatonal que permite solicitar el cruce, atendiendo la petición de forma segura sin interrumpir abruptamente el ciclo vehicular.

Toda la temporización del sistema se maneja con `millis()`, sin utilizar `delay()` en ningún punto del programa, lo que permite que la lectura del botón y la lógica de estados se evalúen de forma continua y sin bloqueos.

## 2. Objetivos de aprendizaje

- Modelar un sistema secuencial como una máquina de estados finitos (FSM) utilizando `enum class`.
- Implementar transiciones de estado controladas por tiempo (`millis()`) y por eventos externos (botón).
- Aplicar una técnica de antirrebote (debounce) por software para la lectura confiable de un botón.
- Diseñar reglas de prioridad y seguridad: la solicitud peatonal solo se atiende en el momento adecuado del ciclo.
- Reforzar el uso de funciones para separar responsabilidades: aplicar salidas, cambiar de estado y leer el botón.

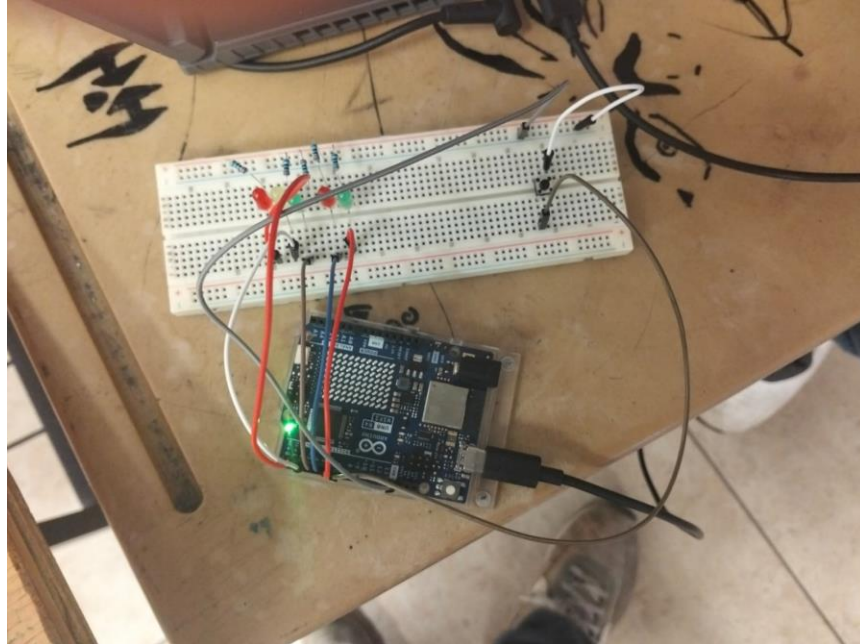
## 3. Material utilizado

- Arduino UNO R4 WiFi
- Protoboard
- LEDs: rojo, amarillo y verde (vehicular) y rojo/verde (peatonal)
- Resistencias limitadoras de corriente para cada LED
- Pulsador (botón) para la solicitud peatonal
- Cables de conexión (jumpers)
- Cable USB para alimentación y programación

## 4. Diagrama de conexión

- `PIN_VEH_ROJO` → Pin digital 2
- `PIN_VEH_AMARILLO` → Pin digital 3
- `PIN_VEH_VERDE` → Pin digital 4
- `PIN_PEA_ROJO` → Pin digital 5
- `PIN_PEA_VERDE` → Pin digital 6
- `PIN_BOTON` → Pin digital 7 (configurado como `INPUT_PULLUP`)

A continuación se muestra el montaje físico utilizado durante la práctica:



## 5. Análisis del código

### 5.1 Máquina de estados finitos (FSM)

El sistema se modela con un enum class llamado EstadoSemaforo, con cuatro estados posibles: VEH\_VERDE, VEH\_AMARILLO, VEH\_ROJO y PEA\_VERDE. El uso de enum class evita fugas de nombres al espacio global y obliga a un tipado fuerte, reduciendo errores al comparar estados.

### 5.2 Transición de estados

```
void cambiarEstado(EstadoSemaforo nuevoEstado) {  
    estadoActual = nuevoEstado;  
    inicioEstado = millis();  
    aplicarSalidasEstado(nuevoEstado);  
}
```

Cada vez que el sistema cambia de estado, se registra el tiempo de entrada (inicioEstado) y se actualizan las salidas correspondientes mediante aplicarSalidasEstado(), la cual primero apaga todos los LEDs y luego enciende únicamente los que pertenecen al nuevo estado. Esta separación evita estados intermedios inconsistentes.

### 5.3 Antirrebote del botón (debounce)

La función botonPresionado() separa dos roles: la última lectura cruda del pin (usada solo para detectar cambios) y el estado ya confirmado como estable. Solo cuando la lectura permanece estable durante más de T\_DEBOUNCE (50 ms) se considera un cambio real, y únicamente se reporta una pulsación válida en el flanco de bajada (transición a LOW), evitando falsos positivos por rebote mecánico del botón.

### 5.4 Reglas de la solicitud peatonal

La solicitud de cruce peatonal solo se "arma" (solicitudPeatonal = true) si el botón se presiona mientras el semáforo vehicular está en VEH\_VERDE o VEH\_AMARILLO. Si se presiona en otro momento, se ignora. Una vez armada, la solicitud se atiende en cuanto el semáforo vehicular llega a VEH\_ROJO, transicionando de inmediato a PEA\_VERDE. Si nadie solicita el cruce, el ciclo vehicular continúa por sí solo una vez transcurrido su tiempo en rojo.

## 6. Tiempos configurados

- Vehicular verde: 8000 ms
- Vehicular amarillo: 2000 ms
- Vehicular rojo (sin solicitud): 6000 ms
- Peatonal verde: 5000 ms
- Antirrebote del botón: 50 ms

## 7. Conclusiones

Esta práctica permitió aplicar el patrón de máquina de estados finitos a un problema real de control secuencial, combinando temporización no bloqueante con `millis()` y eventos externos asíncronos (el botón peatonal). Se reforzó la importancia de separar la lectura de entradas (con antirrebote), la lógica de transición y la aplicación de salidas en funciones independientes, lo cual hace el código más legible, mantenible y fácil de extender. Además, se comprendió cómo diseñar reglas de seguridad y prioridad dentro de una FSM, garantizando que una solicitud externa (el peatón) se atienda en el momento correcto del ciclo sin comprometer la lógica vehicular.

## 8. Video del funcionamiento

Se anexa video demostrativo mostrando el ciclo completo del semáforo (verde → amarillo → rojo) y su comportamiento ante la solicitud peatonal.